Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Genel Konular

- Dersin kapsamı ve kaynakları (ekrandan, 00:10:00 ve 00:20:00)
  - Ders iki kısımdan oluşur: Kısım 1 Gang of Four (Dörtlü Çete) tasarım kalıpları, Kısım 2 Code Smells and Refactoring (birim sınamaları ile).
  - Tasarım kalıpları nesneye dayalı tasarım ve modellemenin nasıl doğru yapılacağını anlatır. Kod kusurları ise modellemede yapılabilecek yanlışları ve bunları düzeltmenin yollarını gösterir; hoca bunu "hatalardan öğrenme" diye tanımladı.
  - Kısım 1'in temel kaynağı: *Design Patterns: Elements of Reusable Object-Oriented Software*, Erich Gamma vd. (Gang of Four), Addison-Wesley, 1994. Kitapta 24 kadar kalıp anlatılır; derste Interpreter dışındaki bütün kalıplar işlenecektir (hoca Interpreter'ın uygulanabilirliğini düşük buluyor).
  - Kısım 2'nin kaynağı: *Refactoring: Improving the Design of Existing Code*, Martin Fowler, Addison-Wesley, 1999. Bütün kod kusurları değil, daha az çabayla daha çok yarar getiren, yaklaşık yarısını oluşturan bir alt küme işlenecektir.
  - Ek kaynaklar (slayttaki liste): *Design Patterns Explained* (Shalloway, Trott), *Head First Design Patterns* (Freeman vd.), *Design Patterns* (Lasater), *Design Patterns Java Workbook* (Metsker), *Applied Java Patterns* (Stelting, Maassen).
  - GoF kitabı kalıbı kuramsal olarak ve UML sınıf şeması üzerinden anlatır. Ek kitaplar ise kalıbın bir problemde nasıl uygulandığını ve diğer çözümlerden neden daha iyi olduğunu gösterir. Hocanın anlatımı da "bir örnekte bir kalıp kullanma" biçimindedir.
- Ön koşullar (ekrandan, 00:20:00)
  - Herhangi bir güncel nesneye yönelik programlama diline hakimiyet (Java, C++, C#). Kod örnekleri Java ile verilir; dile özgü yetenekler kullanılmaz.
  - Kalıtım ve soyut sınıflar en temel gereksinimdir. Doğru kurulan kalıtım hiyerarşisi güçlü bir araçtır, yanlış kurulan hiyerarşi çok şey kaybettirir.
  - UML sınıf şemalarına hakimiyet: şemayı çizebilmek, şemadan kod yazabilmek, koddan şema çıkarabilmek. Sequence diyagramları da yararlıdır. Önerilen kaynaklar: *UML Distilled* (Martin Fowler, 3. baskı, 2003) ve *UML 2.0 in a Nutshell* (Pilone, Pitman).
- Yazılım kalitesi (ekrandan, 00:40:00 ve 00:41:40)
  - Amaç "iyi" tasarım ile "kaliteli" yazılıma ulaşmaktır.
  - Slayttaki tanım: problem alanı hangi düzeyde parçalara ayrılabiliyorsa o düzeyde parçalara ayrılabilen tasarım "iyi" sayılır. Hoca bu tanımı öznel buldu, daha nesnel ölçütler gerektiğini söyledi.
  - Dış kalite özellikleri kullanıcıya yöneliktir: doğruluk, etkinlik, kullanım kolaylığı.
  - İç kalite özellikleri programcıya yöneliktir: yeniden kullanılabilirlik, ilgi alanlarının ayrılması, esneklik, taşınabilirlik, okunabilirlik ve anlaşılabilirlik, sınanabilirlik, bakım kolaylığı. Tasarım kalıpları iç kalite özelliklerini iyileştirir.
- Neden iyi tasarım? (ekrandan, 00:48:20, 00:50:00 ve 00:52:50)
  - Yazılım projeleri önemli oranda başarısız olur. Etken çoktur, ama "tasarımın iyiliği" denetlenebilen bir etkendir.
  - Hatayı düzeltmenin bedeli hayat döngüsünde ilerledikçe üstel olarak artar; geliştirme çabasının çoğu bakım aşamasında harcanır.
  - Katı başarı tanımı üç ayaklı bir sehpaya benzer: belirlenen süre, belirlenen bütçe ve üzerinde anlaşılmış gereksinimlerin tamamı. Bir ayak bile tutmazsa proje başarısızdır.

  - Dr. Dobb's Journal 2013 IT Project Success Survey: geleneksel (şelale) ve ad-hoc süreçlerde tam başarı yaklaşık %50, çevik ve yinelemeli süreçlerde yaklaşık %65, Lean'de biraz daha yüksektir.
  - Türkiye'de çevik projeler üzerine 35 katılımcılı bir çalışmada (UYMS 2014): projelerinin en az %75'i başarılı olan kurum oranı %31, en az %50'si başarılı olan kurum oranı %68'dir.
- Bağlaşım (coupling) (ekrandan, 00:54:10, 01:10:00, 01:20:00, 01:30:00)
  - İyi tasarımın iki temel ilkesi düşük bağlaşım (low coupling) ve yüksek uyumdur (high cohesion). Üçüncü ilke olan ilgi alanlarının ayrılması (separation of concerns) ilk ikisiyle açıklanabilir.
  - Bağlaşım, bir parçanın (sınıf, alt sistem, paket) diğer parçalara bağımlılık oranıdır.
  - Bir sınıf başka bir sınıfın hizmetlerini kullanıyorsa, onun iç yapısını ya da çalışma ilkesini bilmesi gerekiyorsa ya da aralarında kalıtım varsa ona bağımlıdır.
  - Bağlaşım sayılırken ilişkinin varlığı önemlidir, niceliği (çokluk göstergesi) önemli değildir. Okların yönü ise önemlidir: karşılıklı bağımlılık iki ilişki sayılır.
  - A→B→C zinciri ile A→B, A→C yıldızı karşılaştırıldı: toplam bağlaşım, maksimum bağlaşım ve dolaylı ilişkiler farklı sonuçlar verdi.
- Uyum (cohesion) (ekrandan, 01:40:00, 01:50:00, 01:51:10)
  - Uyum, bir parçanın sorumluluklarının birbiriyle uyumlu olma derecesidir.
  - Veritabanı işlemlerini ayrı bir sınıfa vermek bağlaşımı artırır, ama uyumu artırır ve katmanlı bir yapı (iş mantığı katmanı ve veritabanı katmanı) oluşturur.
  - Uyum, sınıf içindeki metot-alan erişim deseninden ölçülebilir: gerçekleşen erişim sayısı / olası en çok erişim sayısı.
- Tasarım kalıbı kavramı (ekrandan, 02:10:00, 02:13:10, 02:20:00)
  - Tasarım kalıbı, yazılım tasarımında karşılaşılan belirli sorunların yalın ve güzel çözümlerinin tarifidir. Somut bir gerçekleme olmak zorunda değildir, soyut bir tarif de olabilir.
  - Amaç tekerleği yeniden keşfetmeden iyi tasarıma ulaşmak, sorunları esnek ve yeniden kullanılabilirliği artıracak biçimde çözmektir.
  - Dört ana bileşen: isim, sorun, çözüm, tartışma/sonuçlar.
  - GoF kalıpları amaçlarına göre üç kümeye ayrılır: yaratımsal (creational), yapısal (structural), davranışsal (behavioral). Ayrıca kapsamlarına göre sınıfsal ve nesnesel diye de ayrılır.
- Örnek tasarım kalıbı: Model-View-Controller (MVC) (ekrandan, 02:30:00, 02:31:10, 02:40:00, 02:44:10)
  - Sorun: aynı veriyi farklı biçimlerde göstermek ve kullanıcıya verinin nasıl ve ne kadarının gösterileceğini seçme olanağı vermek.
  - Çözüm: veriyi (Model), verinin gösterimini (View) ve gösterimin yönetimini (Controller) ayrı sınıflar olarak ele almak.
  - Asgari ilişkiler: Controller→View, Controller→Model, View→Model. Model kimin onu gösterdiğini ya da denetlediğini bilmez.
  - Çeşitlemeler: Model'in değişikliklerini View'a bildirmesi (Observer ile), birden fazla view (iç içe, ayrı görevcikte, örtüşen/ayrık), birden fazla kontrolcü (klavye, fare, ses).

Hocanın Özellikle Vurguladığı Kısımlar

- Nesneye yönelik programlama ve UML sınıf şeması bilgisi şarttır
  - Bir işin nasıl iyi yapılacağını öğretebilmek için önce o işin nasıl yapılacağını bilmek gerekir. Tasarım yapmış, kodlamış ve hata ayıklamış olmak bu dersin temelidir.
  - Hocaya göre "senior" olmak gerekmez. Nesneye yönelik programlama ders notlarının (istisna yönetimi dahil) ilk yarısını sıkıntısız bilmek ve birkaç ay kod geliştirmiş olmak yeterlidir.
- Yeniden kullanımdan beklenen kazanç sınırlıdır
  - Okuduğu araştırmalara göre yeniden kullanımın getirdiği kazanç en fazla %30'lara çıkar; %50-60 beklenmemelidir. Neden: yeniden kullanılacak kod daha fazla çabayla yazılır, ilk maliyet yüksek olur.
  - Yine de "kâr kârdır"; yeniden kullanım fırsatları sürekli izlenmelidir. Örnek: finans yazılımında döviz kurlarının, sağlık yazılımında hastanın kritik değerlerinin (örneğin kan değerleri) bir eşiğin ya da bandın dışına çıkınca alarm vermesi aynı yapıdır.
- Problem alanına uygun modelleme ölçütlerden önce gelir

  - Hangi tasarım seçilirse seçilsin, gerçek problem alanındaki ilişkilere uymayan bir model kullanılmamalıdır. Bağlaşım tartışması ancak tasarımcının serbestçe karar verebildiği (örneğin altyapı katmanındaki) sınıflar için geçerlidir.
- Bağlaşım ve uyum genelde birlikte gider, çatıştıklarında katmanlı çözüm seçilir
  - Çoğu durumda düşük bağlaşım yüksek uyumla birlikte gelir; bu yüzden bağlaşıma göre karar vermek uyuma göre de karar vermek olur.
  - Az sayıda ama önemli durumda ikisi çatışır. O zaman hangi tasarım daha katmanlı ve derli toplu bir çözüm veriyorsa o seçilir; burada uyuma daha çok dikkat edilmelidir.
- Benzer görünen kalıpların ayırt edici noktaları
  - Doğru kalıbı seçebilmek için yüzeysel olarak benzer, ama nitelik olarak farklı kalıpların ayırt edici noktaları iyi bilinmelidir. Hoca bunları yeri geldikçe vurgulayacağını söyledi.
- Kalıplar arası ilişki şemasının ezberlenmesi gerekmez
  - GoF kitabındaki ilişki şeması yalnızca kalıpların ne ölçüde birbirine bağlı olduğu konusunda görsel bir fikir verir.
- Her çözümün bir bedeli vardır
  - Tartışma/sonuçlar bölümü kalıbın son bileşeni olsa da önemsiz değildir. Mükemmel çözüm yoktur; zaman-bellek çelişkisinde olduğu gibi her iyileştirme bir yerden ödünle gelir.

Kısa Tekrar Notları

- Ders: Kısım 1 GoF tasarım kalıpları (Interpreter hariç), Kısım 2 kod kusurları ve refactoring (seçilmiş alt küme).
- İyi tasarım → kaliteli yazılım. Dış kalite kullanıcıya, iç kalite programcıya yöneliktir; kalıplar iç kaliteyi iyileştirir.
- İç kalite: yeniden kullanılabilirlik, ilgi alanlarının ayrılması, esneklik, taşınabilirlik, okunabilirlik/anlaşılabilirlik, sınanabilirlik, bakım kolaylığı.
- Yeniden kullanım kazancı en fazla yaklaşık %30'dur.
- Katı başarı: süre + bütçe + gereksinimlerin tamamı. Dobb's 2013'e göre geleneksel süreçlerde yaklaşık %50, çevikte yaklaşık %65.
- Hatayı düzeltmenin bedeli hayat döngüsünde üstel artar; çabanın çoğu bakımda harcanır.
- Temel ilkeler: düşük bağlaşım, yüksek uyum; ilgi alanlarının ayrılması bu ikisinden çıkar.
- Bağlaşımda ilişkinin varlığı ve yönü sayılır, niceliği sayılmaz. Dolaylı ilişkiye daha düşük ağırlık (örneğin 0,5) verilebilir.
- Sistem düzeyinde iki ölçüt: toplam bağlaşım ve maksimum bağlaşım (minimaks yaklaşımı). Hangisinin esas alınacağı, geçmiş projelerin başarı verisiyle karşılaştırılarak kararlaştırılabilir.
- Uyum = gerçekleşen metot-alan erişimi / (alan sayısı × metot sayısı). Sınıfı sorumluluklarına göre bölmek uyumu artırır.
- Refactoring: dışarıya verilen hizmetleri değiştirmeden iç tasarımı iyileştirmek.
- Tasarım kalıbının dört bileşeni: isim, sorun, çözüm, tartışma/sonuçlar.
- Kalıpların yararları: tekerleği yeniden icat etmemek, deneyimli tasarımcıların emeğiyle kaliteli çözüm, deneyimin yazılı aktarımı, ekip içinde ortak sözlük.
- Kümeler: yaratımsal, yapısal, davranışsal. Adapter hem sınıfsal hem nesnesel sayılır.
- MVC: Model ham veri, View gösterim, Controller kullanıcı komutları. İlişkiler: C→V, C→M, V→M. M→V bildirimi gerekirse Observer kullanılır.

Detaylı Açıklamalar

- Dersin anlatım yöntemi
  - Hoca her kalıbı önce GoF kitabından alıntılarla tanıtır, sonra bir problemle örnekler. Mümkün olduğunda örnek üç çözümle verilir. Birincisi akla ilk gelen çözümdür. İkincisi, gereksinimde birinci çözümle karşılanması verimsiz olan bir değişiklik geldiğinde nesneye yönelimin ana kurallarına uygun yapılan çözümdür. Üçüncüsü uygun kalıbın eklendiği çözümdür. İkinci ile üçüncünün karşılaştırılması kalıbın kazancını gösterir.
  - Ders notlarındaki kalıpların bir kısmının örnekleri ek kitaplardan alınmış, yeterli örnek bulunamayan kalıplar için örnekleri hoca kendisi hazırlamıştır.

- İç kalite özelliklerinin açıklaması
  - **İlgi alanlarının ayrılması:** Tasarım bileşenlerinin odaklanmış sorumluluklara sahip olması gerekir. Böylece gereksinim değişikliklerinden daha az etkilenirler ve benzer gereksinimli başka projelerde yeniden kullanılma olasılıkları artar.
  - **Esneklik:** Gereksinimler mutlaka değişir; kurallar, yönetmelikler ve pazar şartları değişir. Esnek yazılmış kod bu değişikliklere daha rahat uyar, yoksa projeye yeniden başlamak gerekir. Kalıpların önemli bir vaadi esnek tasarımdır.
  - **Taşınabilirlik:** Ürün ne kadar farklı platforma hitap ederse müşteri kitlesi o kadar genişler.
  - **Okunabilirlik ve anlaşılabilirlik:** Bakım masraflarını düşük tutmanın en önemli etkenlerinden biridir. Yazılımı geliştiren ekip ile sahada bakımını yapan ekip genelde farklı insanlardır; kodun yalnız okunabilir değil, anlaşılabilir de olması gerekir. Tasarım kalıpları meslekte ortak bir sözlük oluşturduğu için anlaşılabilirliği de iyileştirir.
- Bağlaşım örneği: zincir mi, yıldız mı? (ekrandan, 01:20:00 ve 01:30:00)
  - Tasarım I: A→B, B→C. Tasarım II: A→B, A→C. Derste yapılan oylamada 5 kişi I'i, 9 kişi II'yi daha düşük bağlaşımlı buldu.
  - Doğrudan bağlaşım: I'de bağ(A) = 1, bağ(B) = 1, bağ(C) = 0, toplam 2, maksimum 1. II'de bağ(A) = 2, bağ(B) = 0, bağ(C) = 0, toplam 2, maksimum 2. Toplam eşit, maksimuma göre I daha iyi.
  - Maksimum bağlaşıma bakmak oyun kuramındaki minimaks yaklaşımına benzer: olası en kötü senaryolar listelenir, en az zarar riskli olan seçilir.
  - Dolaylı ilişki: I'de A, B üzerinden C'ye dolaylı olarak bağlıdır. Bu ilişkiye 0,5 ağırlık verilirse bağ(A) = 1,5, toplam 2,5, maksimum 1,5 olur. Artık toplamda II, maksimumda I avantajlıdır.
  - 0,5 ağırlığın gerekçesi: C'deki bir değişikliğin B'yi doğrudan etkileme olasılığı, A'ya kadar taşınma olasılığından daha yüksektir.
  - Hangi ölçütün esas alınacağına karar vermek için geçmiş projeler incelenebilir. Her proje için bir başarı ölçütü belirlenir (örneğin süre-bütçe-işlev ayaklarından kaçının tuttuğu ya da toplam hata düzeltme çabası). Sonra başarılı projelerde hangi ölçütün daha iyi çıktığına bakılır. Bunun için proje kayıtlarının iyi tutulması ve ölçütleri otomatik hesaplayan araçlar gerekir.
- Uyum örneği: veritabanı sorumluluğu (ekrandan, 01:40:00)
  - Tasarım I'de A, B, C sınıflarının her biri kendi CRUD (oluşturma, okuma, güncelleme, silme) işlemlerini veritabanıyla kendisi yapar. Tasarım II'de bu işler ayrı bir VT sınıfında toplanır; VT sınıfı A, B, C'ye bağımlıdır. Oylamada 1 kişi I'i, 13 kişi II'yi seçti.
  - II bağlaşımı artırır, ama A, B, C yalnızca iş mantığıyla uğraşır, VT sınıfı yalnızca kalıcılıkla uğraşır. Bu, iş mantığı katmanı ile veritabanı katmanından oluşan katmanlı bir yapı demektir.
  - Veritabanı değiştiğinde (örneğin ilişkisel bir veritabanından MySQL'e ya da NoSQL bir veritabanına geçildiğinde) yalnızca VT sınıfı değişir, iş mantığı sınıfları etkilenmez.
  - Hocanın benzetmesi: VT sınıfı aynı işi beş ya da elli sınıf için yapsa da yaptığı iş türü değişmez. Bir babanın bir yerine beş çocuğu olması gerçekten farklı sorumluluklar getirir, ama yazılımda aynı kod aynı işi çok kez yapar.
  - Başka benzetmeler: test mühendisine aynı anda geliştirici görevi de verilirse hangi sorumluluğu hangi sırada yerine getireceğini şaşırır. Montaj hattında her işçinin kendi uzmanlaştığı işi yapması, tek işçinin her şeyi yapmasından daha hızlıdır.
- Uyum ölçümü: metot-alan erişim deseni (ekrandan, 01:50:00 ve 01:51:10)
  - Sınıfta 3 alan (a1, a2, a3) ve 4 metot (m1-m4) var. Erişimler: m1→a1, m1→a2, m2→a1, m2→a2, m3→a2, m4→a3. Toplam 6 erişim.
  - Olası en çok erişim $3 \times 4 = 12$ olduğundan uyum $6/12 = 0{,}5$, yani %50'dir.
  - Sınıf ikiye bölünür: Sınıf I = {a1, a2; m1, m2, m3}, Sınıf II = {a3; m4}. Sınıf I'de olası en çok erişim $2 \times 3 = 6$, gerçekleşen 5, uyum $5/6 \approx 0{,}83$. Sınıf II'de $1/1 = 1$.
  - Ortalama uyum $(0{,}833 + 1)/2 \approx 0{,}917$, yani yaklaşık %92 olur; tek sınıftaki %50'den çok daha yüksektir.
  - Ders sonrası not: Hoca ortalamayı $183/2 = 91{,}5$ diye hesapladı; %83 yuvarlanmış değerdir. Tam değerle sonuç $(5/6 + 1)/2 \approx \%91{,}7$'dir.
  - Mevcut tasarımı dışarıya verilen hizmetleri değiştirmeden iç tasarımı değiştirerek iyileştirmeye refactoring denir; dersin son haftalarının konusudur. Uyumu hesapla-

manın kod yorumlarından ya da makine öğrenmesinden yararlanan başka yöntemleri de vardır.

- Tasarım kalıbının bileşenleri (ekrandan, 02:10:00)
  - **İsim:** Kalıbı en iyi anlatan bir ya da birkaç kelimedir. Kalıp özümsendikten sonra yalnızca adı söylendiğinde öteki üç bileşeni hatırlatabilmelidir.
  - **Sorun:** Kalıbın çözdüğü sorunun tarifidir.
  - **Çözüm:** Önerilen tasarımın bileşenleri, bunların ilişkileri, sorumlulukları ve birlikte çalışmalarıdır. UML sınıf şeması, gerekirse başka şemalar ve kodla anlatılır.
  - **Tartışma/Sonuçlar:** Çözümün esneklik, genişletilebilirlik, taşınabilirlik gibi yönlerden sisteme etkileri; güçlü ve zayıf yönleri; yer-zaman çelişkileri ve başka kalıplarla işbirliği olanakları.
- Kalıplar arası ilişkiler (ekrandan, 02:13:10)
  - GoF kitabındaki şemada örneğin Template Method'un adımları Strategy ile tanımlanabilir ("defining algorithm's steps"). Hoca şemada olmayan bir ilişki de ekledi: bazı Strategy gerçeklemeleri Singleton olabilir.
  - Bir kalıbı kullanırken problemin başka kısımlarına hangi kalıpların eklenebileceğini bilmek (örneğin Composite'in çocuklarını dolaşmak için Iterator) bu şemanın yararıdır.
- Kalıpların yararları ve ortak sözlük
  - Örnek: Kıdemli tasarımcılar, sistemdeki farklı optimizasyon algoritmalarının her birini Strategy olarak gerçeklemeyi önerir. Alt adımları zamanla değişebilecek algoritmalar için de Decorator önerilir. Kalıp adları sayesinde önemli tasarım kararları birkaç kelimeyle aktarılır; kalıpları bilen ekip aynı sözlüğü kullanır.
  - Teknik olmayan bir proje yöneticisinin kalıpları bilmesi gerekmeyebilir, ama işin teknik kısmına dokunan herkesin bilmesinde çok yarar vardır.
- Kalıpların sınıflandırılması ve anlatım sırası (ekrandan, 02:20:00)
  - **Yaratımsal (creational):** Nesnelerin oluşturulması, temsili ve ilişkilendirilmesindeki bağımlılığı azaltmaya yönelik kalıplar.
  - **Yapısal (structural):** Sınıflar ve nesnelerin bir araya getirilerek daha büyük yapılar elde edilmesine yönelik kalıplar.
  - **Davranışsal (behavioral):** Sorumlulukların nesnelere dağıtılması ve algoritma seçimine yönelik kalıplar.
  - Sınıfsal kalıplar sınıflar ve alt sınıflar arasındaki kalıtım ilişkilerine odaklanır. Nesnesel kalıplar çalışma anında kurulan ve değişen nesne ilişkilerine, yani daha dinamik bir bakışa odaklanır. Adapter iki gruba da konur.
  - GoF kitabında bir de tasarım problemine göre gruplama vardır: uygun nesneleri bulmak (Composite, Strategy, State gibi), nesne taneciklilik düzeyini belirlemek, arayüzleri belirlemek (Memento, Decorator, Proxy, Visitor gibi). Hoca birkaç yıl bu sırayla anlatmış, sonra henüz görülmemiş kalıplara çok fazla atıf gerektiği için kitabın yaratımsal-yapısal-davranışsal sırasına dönmüş.
- MVC kalıbı ayrıntıları (ekrandan, 02:31:10 ve 02:44:10)
  - **Bağımlılıkların gerekçesi:** Model yalnızca veridir; public metotlarını kim çağırırsa ona hizmet verir. View, çizim için gereken verileri alabilmek için hangi modelle çalıştığını bilmelidir. Controller hem modeli hem view'u bilmelidir, çünkü view oluştururken ona hangi veriyle çalışacağını verir.
  - **Adın kaynağı (ressam benzetmesi):** Resmi yapılan kişi modeldir. Boy resmi, portre, yağlıboya, karakalem ya da karikatür gibi farklı bakış açıları view'lardır. Sanatsal kararları vererek ilgili view'dan yola çıkıp modeli çizen ressam ise kontrolcüdür.
  - **Yararlar:** Gösterim, kontrol ve veri ayrıldığı için bunlar çalışma anında değiştirilebilir, ilgi alanları ayrılır, bağlaşım düşer, uyum ve yeniden kullanım artar.
  - **Model→View bildirimi:** Veri durağan değilse (örneğin seçim gecesi sandıklar açıldıkça değişen oy oranlarının çubuk grafik, pasta grafik ya da harita üzerinde gösterilmesi) en iyi yanıt süresi için Model değişiklikleri View'a bildirmelidir. Bunun için Model→View yönünde bir ok eklenir ve Observer kalıbı kullanılabilir. Bu ok eklenmezse View, Model'i belirli aralıklarla sorgular. Sık sorgulama ağ trafiğini artırır (özellikle model ile view farklı makinelerdeyse), seyrek sorgulama güncelliği düşürür. Doğru denge problem alanına göre seçilir.
  - **Birden fazla view:** Görünümler iç içe olabilir (ilçe-il-bölge-ülke oy oranları), ayrı görevcikte (thread) çalışabilir ya da örtüşen/ayrık olabilir (PDF'de bir ekranda önceki

sayfanın sonu ile sonraki sayfanın başının birlikte görünmesi). Bir model aynı anda birden fazla view tarafından gösteriliyorsa ilişki bire-bir değil bire-çok olur ve dizi, liste ya da ağaç gibi bir veri yapısıyla tutulması gerekir.
  - **Birden fazla kontrolcü:** Girdi klavyeden, fareden ya da sesle dikteyle gelebilir. Kontrolcüler arasında çalışma anında geçiş istenirse Strategy kalıbı kullanılabilir.
  - MVC deyince Microsoft'un MVC çatısı akla gelebilir. MVC'nin ilk ve özgün kullanımı bir tasarım kalıbı olarak kullanımıdır; Microsoft'un çatısı bu kalıbı .NET projelerinde kolayca kullanmayı sağlar.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.